





# **SESIÓN 12: SWING**

**Docente del Curso**



## Reflexión



Puedes ver el video:

[https://www.youtube.com/watch?v=B6q4Bk3trPM&ab\\_channel=WarriorTVSA](https://www.youtube.com/watch?v=B6q4Bk3trPM&ab_channel=WarriorTVSA)



Al terminar la sesión el estudiante desarrolla y expone casos de la programación orientada a objetos aplicando SWING, mediante el uso del lenguaje de programación de Java y con la asesoría brindada por el docente.



# ¿Qué son los Componentes Swing? Una Introducción

## Definición

Swing es una biblioteca de widgets GUI para Java. Forma parte de las Java Foundation Foundation Classes (JFC) y proporciona un conjunto de componentes ligeros que se ejecutan de manera uniforme en todas las plataformas.

## Ventajas

Swing ofrece una mayor flexibilidad y personalización que AWT (Abstract Window Toolkit), ya que los componentes se dibujan directamente en la pantalla en lugar de depender de los componentes nativos del sistema operativo. Esto garantiza una apariencia coherente en diferentes plataformas.

## Importancia

Los componentes Swing son fundamentales para el desarrollo de aplicaciones de escritorio en Java, permitiendo crear interfaces de usuario interactivas y ricas en funcionalidades. Su arquitectura basada en componentes facilita la reutilización y el mantenimiento del código.



1

## JComponent

Es la clase base para todos los componentes Swing. Define propiedades y métodos comunes como color de fondo, fuente y comportamiento del componente.

2

## Contenedores

Son componentes que pueden contener otros componentes. Los principales contenedores son JFrame (ventana principal), JPanel (panel genérico) y JDialog (ventana de diálogo).

3

## Componentes Visuales

Son los componentes que se muestran al usuario, como JButton (botón), JLabel (etiqueta), JTextField (campo de texto) y JTable (tabla).

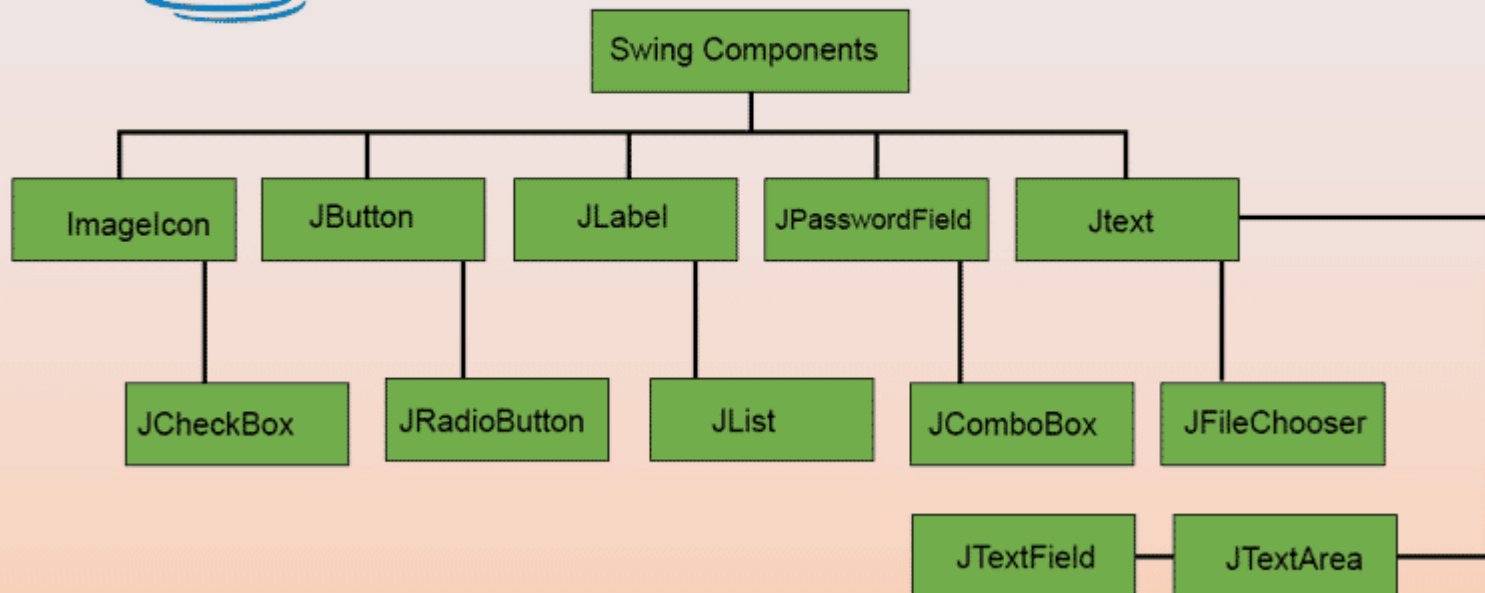


SWING es una biblioteca de componentes gráficos para Java que permite la creación de interfaces de usuario (UI) para aplicaciones de escritorio.

Proporciona un conjunto de clases y métodos que permiten diseñar y desarrollar interfaces gráficas de usuario de manera fácil y eficiente.



# Swing Components in Java





## ELEMENTOS DE LA LIBRERÍA SWING



**JFrame:** Es el marco principal de una aplicación SWING. Se utiliza para crear la ventana principal de la aplicación.

```
import javax.swing.*;

public class MiVentana extends JFrame {
    public MiVentana() {
        // Configurar la ventana
        setTitle("Mi Aplicación");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Agregar otros componentes aquí
    }

    public static void main(String[] args) {
        // Crear una instancia de la ventana y mostrarla
        MiVentana ventana = new MiVentana();
        ventana.setVisible(true);
    }
}
```



**JPanel:** Es un contenedor que se utiliza para organizar y manejar otros componentes.

```
import javax.swing.*;

public class MiPanel extends JPanel {
    public MiPanel() {
        // Configurar el panel
        // Agregar otros componentes aquí
    }
}
```



**JButton:** Es un botón que se utiliza para realizar acciones cuando se hace clic en él.

```
import javax.swing.*;

public class MiVentana extends JFrame {
    public MiVentana() {
        // Configurar la ventana
        JButton boton = new JButton("Haz clic aquí");
        boton.addActionListener(e -> {
            // Acciones al hacer clic en el botón
            JOptionPane.showMessageDialog(this, "¡Botón presionado!");
        });
        add(boton);
    }
}
```



JLabel: Es un componente utilizado para mostrar texto o imágenes.

```
import javax.swing.*;

public class MiVentana extends JFrame {
    public MiVentana() {
        // Configurar la ventana
        JLabel etiqueta = new JLabel("¡Hola, mundo!");
        add(etiqueta);
    }
}
```



**TextField:** Es un campo de texto donde el usuario puede ingresar texto.

```
import javax.swing.*;

public class MiVentana extends JFrame {
    public MiVentana() {
        // Configurar la ventana
        JTextField textField = new JTextField(20);
        add(textField);
    }
}
```

# Layout Managers: Posicionamiento de Componentes en la Interfaz

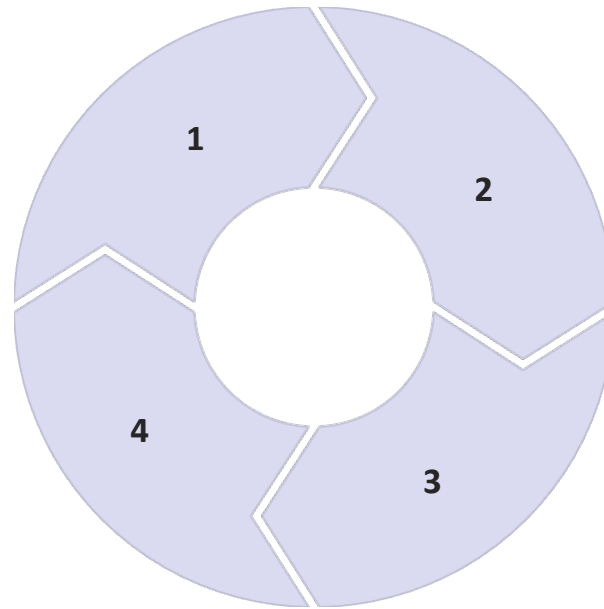


## FlowLayout

Posiciona los componentes en una fila, uno tras otro. Si no hay suficiente espacio, los componentes se trasladan a la siguiente fila.

## BoxLayout

Organiza los componentes en una sola fila o columna. Permite especificar la alineación y el espaciado entre los componentes.



## BorderLayout

Divide el contenedor en cinco regiones: NORTH, SOUTH, EAST, WEST y CENTER. Cada región puede contener un componente.

## GridLayout

Organiza los componentes en una cuadrícula con un número específico de filas y columnas. Todos los componentes tienen el mismo tamaño.

# Programación de Componentes Gráficos Personalizados



- 1 Extender JComponent
- 2 Override paintComponent()
- 3 Añadir propiedades

Para crear un componente Swing personalizado, se debe extender la clase JComponent e implementar el método paintComponent().

En este método, se dibuja el componente utilizando las clases Graphics y Graphics2D.

También se pueden agregar propiedades y métodos personalizados para controlar el comportamiento del componente.



1

## Listeners

Los listeners son interfaces que definen métodos para responder a eventos específicos. Por ejemplo, `ActionListener` para eventos de clic de botón y `KeyListener` para eventos de teclado.

2

## Adapters

Los adapters son clases que implementan las interfaces de listeners y proporcionan implementaciones vacías para todos los métodos. Se utilizan para implementar solo los métodos que son de interés.

3

## Ejemplo: `ActionListener`

Para agregar un `ActionListener` a un botón, se utiliza el método `addActionListener()`. La clase que implementa la interfaz `ActionListener` debe implementar el método `actionPerformed()`.



## CONCLUSIONES Y PREGUNTAS

